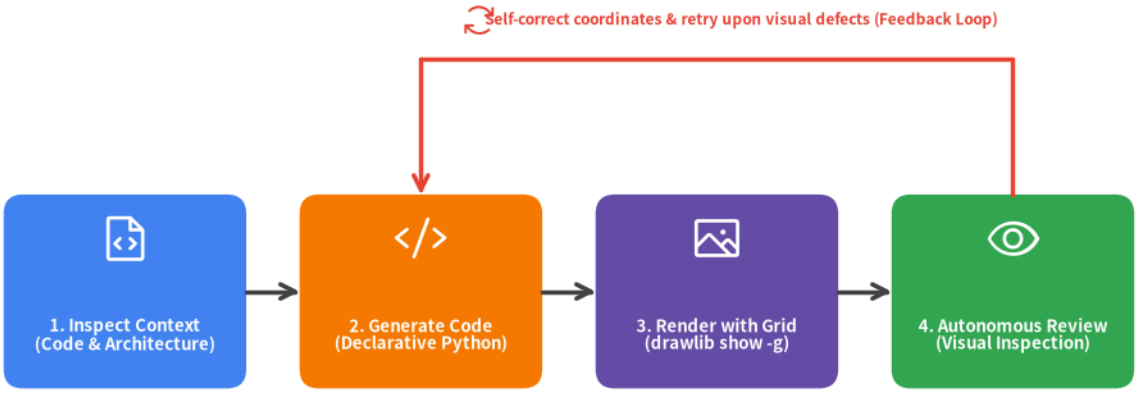
Automated Documentation with Drawlib & AI

Illustrated Documentation as Code in the AI Era



Autonomous AI Visual Self-Correction Loop

Author: Drawlib Core Team

Audience: Software Engineers, Architects, and AI Pair Programmers

Version: 0.3.0

Date: October 2026

Table of Contents

Chapter 1: Challenges in Modern Technical Documentation

Chapter 2: The Solution: Illustrated Documentation as Code

Chapter 3: Drawlib Architecture and Capabilities

Chapter 4: Installation Guide

Chapter 5: Project Scaffolding and Build Workflows

Chapter 6: AI Agent Integration and Prompting

Chapter 7: Drawing Primitives

Chapter 8: SmartArts (Structured Infographics)

Chapter 9: Quantitative Charts

Chapter 10: Technical Diagrams (Architecture & System Design)

Chapter 11: Style and Theme Customization

Chapter 12: Slide Deck Presentations (`slide`)

Chapter 1: Challenges in Modern Technical Documentation

In rapid software development and AI-assisted programming, technical documentation and architectural diagrams face chronic challenges.

1.1 The Stale Architecture Diagram Problem (Diagram Rot)

In many engineering teams, documentation management suffers from the following issues:

- **GUI Drawing Tools are Disconnected from Code**: Diagrams drawn in tools like Figma, draw.io, or Miro are isolated from source code repositories.
- **Binary Image Pollution**: When diagrams are exported as binary PNG files and committed to Git, pull requests cannot review visual or textual diffs.
- **Immediate Obsolescence**: When APIs, data models, or cloud topologies change during feature sprints, diagrams are left un-updated, rapidly turning documentation into obsolete fiction.

1.2 Limitations of Existing Documentation-as-Code (DaC)

While Markdown-based Documentation-as-Code (DaC) is common, existing visual diagram tools introduce significant friction:

- **Mermaid Layout Instability**: Mermaid is convenient for simple flowcharts, but its opaque heuristic layout solver frequently shuffles node positions, breaks edge routing, and provides zero pixel-level control.
- **Lack of Cloud & Icon Assets**: High-level architectural diagrams require official cloud icons (GCP, AWS) and standardized icon sets (Phosphor, FontAwesome), which basic markdown chart plugins lack.
- **Inflexible Styling**: Matching diagram palettes with corporate design tokens or editorial PDF themes is notoriously difficult.

1.3 Why AI Coding Agents Struggle with Manual Drawing

Generative AI coding assistants (Claude, Cursor, Gemini) excel at writing code, but struggle with traditional diagramming:

1. **No GUI Hands**: AI cannot drag and drop shapes in desktop or web canvas interfaces.
2. **Raw SVG Boilerplate is Error-Prone**: Generating hundreds of lines of raw SVG XML or low-level matplotlib boilerplate often causes label clippings and overlapping lines.
3. **No Visual Self-Correction**: Without an explicit coordinate system and image feedback loop, AI models cannot verify whether their diagrams look clean.

Chapter 2: The Solution: Illustrated Documentation as Code

Drawlib delivers **"Illustrated Documentation as Code" (IDaC)** for software engineers and AI coding agents. While traditional Documentation-as-Code focused solely on text, Drawlib elevates architectural diagrams, system designs, and workflows into first-class, version-controlled code.

2.1 Core Principles of Illustrated Documentation as Code

Drawlib is built on four pillars:

1. **Declarative Python Scripts**: Build diagrams using clean, intuitive, high-level Python APIs.
2. **Inline Markdown Embedding**: Embed diagrams directly in your technical documentation via `drawlib` code fences.
3. **Full Version Control & Git Diffs**: Visual definitions live as code inside Markdown files, allowing seamless peer review in pull requests.
4. **Automated Compilation**: Compile everything with a single CLI command (`drawlib build`) into static HTML sites, GitHub-flavored Markdown, and publication-ready PDF books.

2.2 Traditional Approach vs. Drawlib

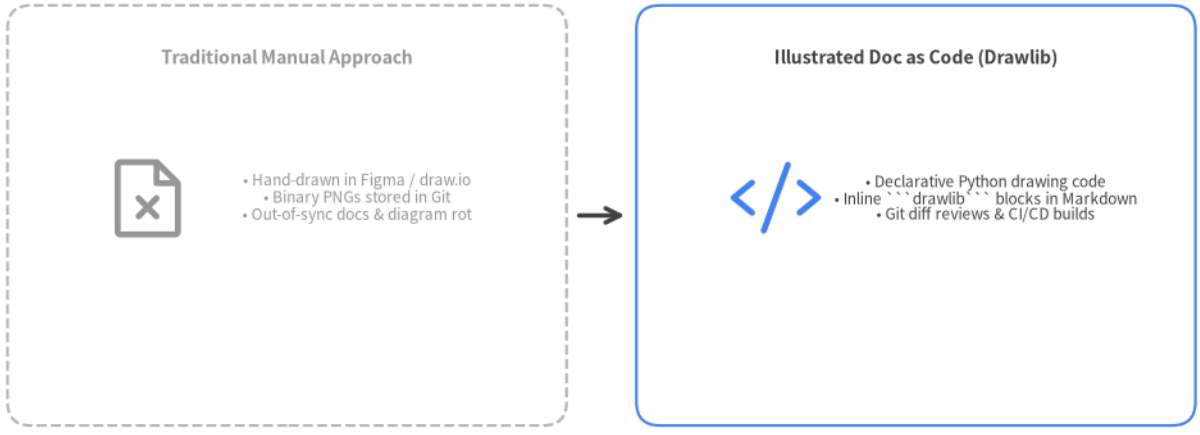


Figure 2.1: Comparing Traditional Manual Drawing with Drawlib

2.3 Synergy with AI Pair Programming

Expressing diagrams as executable code transforms the workflow of AI agents:

- **Spec-Driven Generation**: When you implement a new microservice or API route, prompt your AI: *"Update the architecture diagram in doc.md"*. The agent writes the Python block instantly.
- **Multimodal Visual Verification**: AI agents render images in headless mode, visually inspect them with file tools, and fix overlapping labels before showing them to you.
- **100% Deterministic Reproducibility**: The same code produces identical vector illustrations across all development environments.

Chapter 3: Drawlib Architecture and Capabilities

Drawlib is an integrated platform for Illustrated Documentation as Code. It unifies document building with a 4-layer drawing architecture, ranging from geometric primitives to full system topologies.

3.1 Drawing & Documentation Component Layers

Layer	Primary Modules	Description & Use Cases
1. Primitives	`drawlib.shapes`, `lines`, `text`, `icons`, `images`	23 geometric shapes, orthogonal & curved connectors, rich typography, and Phosphor/FontAwesome/GCP icons
2. SmartArts	`drawlib.smartarts`	Sequential chevrons, comparison tables, radial mindmaps, organizational trees, and circular cycle loops
3. Charts	`drawlib.charts`	Gantt schedules, bar charts, time-series line/area plots, pie charts, scatter plots, and radar charts
4. Diagrams & Graphs	`drawlib.diagrams`, `drawlib.graph`	Cloud VPC architectures, workflow flowcharts, sequence diagrams, ER diagrams, UML classes, and Pure-Python auto-layout solvers
5. Presentation & Animation	`drawlib.slide`, `drawlib.anim`	16:9 presentation slide decks, native APNG & Animated WebP generation
6. Builder & CLI	`drawlib._builder`, `drawlib._cli`	HTML / PDF / Markdown multi-target compilation, live-reload preview server

3.2 Unified Design System and Semantic Colors

With Drawlib, you never waste time tweaking arbitrary RGB hex codes. It provides a cohesive **6-role semantic color palette**:

- `Styles.Primary`: Core system nodes and primary paths (Blue)
- `Styles.Secondary`: External entities, user actors, and clients (Slate / Indigo)
- `Styles.Accent`: Highlighted features and auxiliary components (Amber / Teal)
- `Styles.Muted`: Boundary containers, structural grids, and subtle annotations (Light Gray)
- `Styles.Success` / `Styles.Danger`: Operational statuses, health checks, alerts, and errors

Every color role includes standardized PascalCase design tokens such as `PrimaryFlat` (solid fill), `PrimaryOutline` (bordered), `PrimaryBold` (thick line), and `PrimaryDashed` (dashed border).

Principles of Neutral Typography and Connectors

To maintain high contrast, professional aesthetics, and visual clarity:

- **Neutral Typography**: On light backgrounds, text MUST default to `Styles.Dark` or `Styles.DarkBold`. Inside dark-filled shapes, use `Styles.WhiteBold` or `Styles.Light`. Never apply colors (`Primary`, `Secondary`, `Accent`) to text without explicit semantic necessity.
- **Neutral Connectors**: Standard workflow lines, arrows, and sequential transitions default to `Styles.DarkBold`. Reserve accent and status colors (`Styles.Primary`, `Styles.Accent`, `Styles.Danger`) strictly for critical flow paths, active connections, or error loops.

3.3 Typography and Multilingual Support

Drawlib provides first-class typography out of the box:

- **HTML / PDF Output**: Automatic font stacks featuring Google Sans, Inter, Roboto, and Noto Sans.
- **Canvas Rendering**: Universal font loaders prevent missing glyphs and broken character boxes across platforms.

Chapter 4: Installation Guide

Drawlib is packaged as a pure-Python library and managed with `uv`.

4.1 Prerequisites

- **Python**: Version 3.11 or higher.
- **Package Manager**: uv is highly recommended for speed and deterministic virtual environments.

4.2 Standard Installation (CLI & Standalone Drawing)

To install Drawlib directly from the official Git repository into your project:

```
uv add "drawlib @ git+https://github.com/yuichi110/drawlib.git"
```

4.3 PDF Compilation Installation (Full Documentation Suite)

To build publication-ready PDF documents via headless Chromium, install Drawlib with the `[pdf]` extra and install the Chromium browser binary:

```
# 1. Install Drawlib with PDF dependencies (Playwright)
uv add "drawlib[pdf] @ git+https://github.com/yuichi110/drawlib.git"

# 2. Install the Playwright Chromium binary
uv run playwright install chromium
```

4.4 Verification

Verify that the Drawlib CLI is properly installed:

```
uv run drawlib --version
```

Chapter 5: Project Scaffolding and Build Workflows

Drawlib provides the `drawlib init` command to scaffold purpose-built documentation projects in seconds.

5.1 Project Template Types

Template	Use Case	Primary Artifacts
`doc`	Multi-chapter technical specs, engineering RFCs, and formal reports	`<target>.html`, `<target>.pdf`, `<target>.md`
`site`	Multi-page documentation website with sidebar navigation	`<target>_html/` and `<target>/`
`slide`	16:9 presentation slide deck (web & vector PDF)	`slide/index.html` and `slide.pdf`
`images`	Standalone Python drawing scripts	`<target>/*.png`, `*.webp`

5.2 Creating a Google-Style Document Project

Create a Google-style document project in one command:

```
uv run drawlib init doc report -l en -s google
```

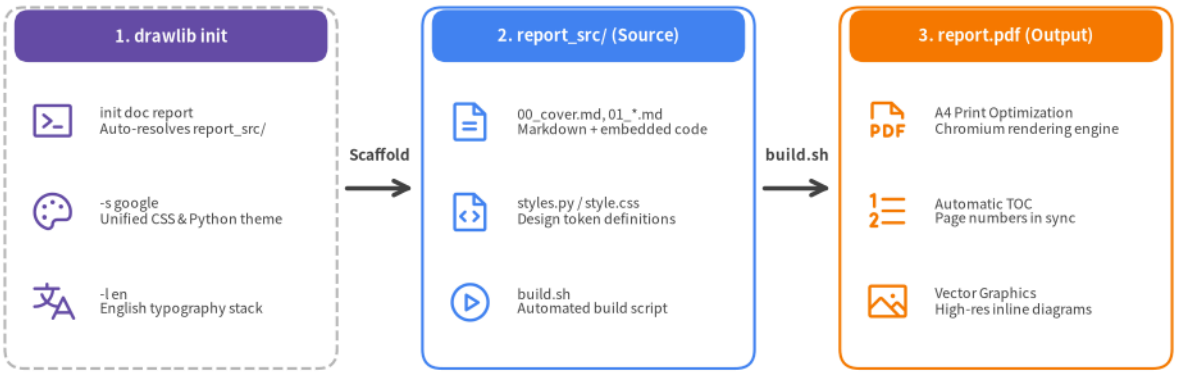


Figure 5.1: Project Lifecycle from Scaffolding to Final Artifact

Automatic Path Resolution via Target Argument

Specifying `drawlib init doc report` establishes a synchronized pipeline:

- **Source Directory**: Creates `report_src/` in current working directory.
- **Build Output**: Generates `report.pdf`, `report_html/`, and `report_markdown/`.

5.3 Directory Structure

The scaffolded `report_src/` folder contains:

```
report_src/
├── 00_cover.md           # [Required] Title page and metadata
├── 01_overview.md        # Chapter 1 content
├── 02_design.md          # Chapter 2 content
├── style.css             # PDF stylesheet (Google Theme)
├── styles.py             # Drawing style definitions (GoogleStyles)
├── template.html         # Jinja2 print layout template
├── utils.py              # Project drawing helpers
├── build.sh              # [Executable] One-click build script
└── README.md             # Project guide
```

5.4 Building the Document

To compile your Markdown documents into a polished PDF, run the generated `build.sh`:

```
# Execute build script
./report_src/build.sh
```

When compilation finishes, `report.pdf` is ready for distribution.

Chapter 6: AI Agent Integration and Prompting

Drawlib is engineered from the ground up to be "Agent-First", enabling AI coding assistants (Claude, Cursor, Gemini, Copilot) to generate accurate, publication-ready diagrams autonomously. Once basic instructions are registered, the AI interacts with the Drawlib CLI to inspect rules, test drawings, and output verified code.

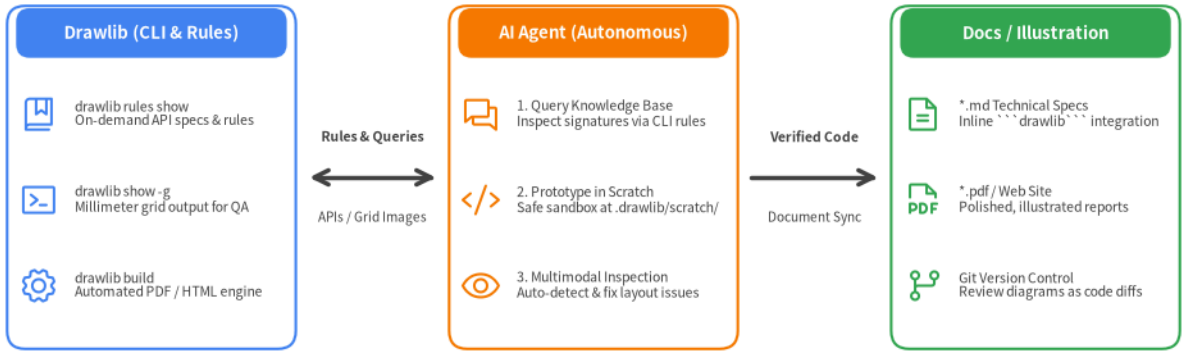


Figure 6.1: Autonomous Interaction Model between Drawlib and AI Agents

6.1 Using the Built-In Rule System (`drawlib rules`)

Drawlib embeds comprehensive API specifications and best practices directly in its CLI. Agents can consult rules on demand:

```
# List all available rule manuals
uv run drawlib rules list

# Read core agent workflow instructions
uv run drawlib rules show agent-instruction

# Check canvas coordinate system rules
uv run drawlib rules show overview

# Check specific domain module specifications
uv run drawlib rules show lib-smartarts
uv run drawlib rules show lib-diagrams
uv run drawlib rules show lib-charts
```

6.2 Setting Project Rule Files

Register these essential rules in `.cursorrules` or `.agents/rules/` so agents follow Drawlib principles automatically:

```
# Drawlib Drawing Rules
- Do NOT generate raw SVG or low-level matplotlib code; always use high-level modules (smartarts, diagrams, charts).
- The canvas origin (0, 0) is at the bottom-left corner.
- Design tokens must be imported and referenced in PascalCase: `Styles` and `Colors`.
- Embedded code blocks must specify `file:<name>.png` and `caption:"..."`.
```

6.3 The Autonomous Visual Feedback Loop

Instruct your agent to execute this 4-step self-correction loop when creating illustrations:

1. **Draft Code**: Write the `drawlib` block in the document.
2. **Render Grid Preview**:

   ```
   uv run drawlib show doc.md arch.png -g -o .drawlib/scratch/arch.png
   ```

3. **Multimodal Review**: Inspect the rendered PNG for label overflow, line overlaps, or tight margins.
4. **Iterative Refinement**: Adjust coordinates and finalize the code.

Chapter 7: Drawing Primitives

Drawlib's primitive drawing layers (`shapes` , `lines` , `text` , `icons`) give you the power to construct custom geometric layouts and specialized diagrams.

7.1 Canvas Coordinate System

Drawlib uses a standard **Cartesian coordinate space**:

- **Origin** `(0, 0)` : Located at the **bottom-left** corner of the canvas.
- **Canvas Dimensions**: Defined explicitly using `setup(width=W, height=H)` .

7.2 Shapes, Lines, and Vector Icons

Over 40 shape primitives (rectangles, circles, ellipses, wedges, shields, ribbons, pins, clocks, block arrows, stars) are built-in:

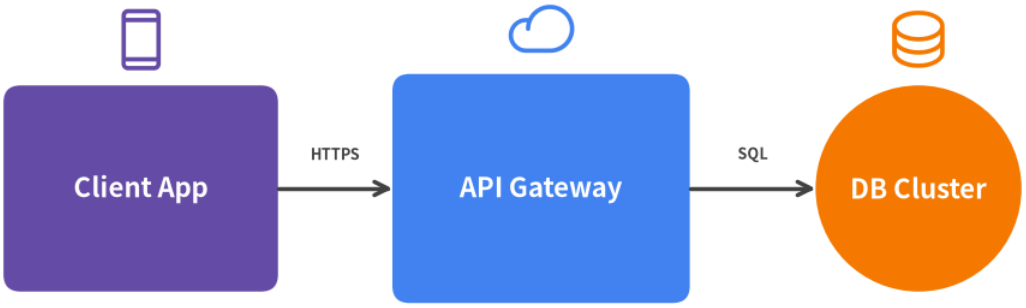


Figure 7.1: Service Architecture Built with Shapes, Lines, and Icons

7.3 Essential Drawing Functions

- `rectangle(xy, width, height, style=...)` : Rectangle (supports rounded corners via `style.shape_r`).
- `circle(xy, radius, style=...)` : Circle.
- `line(start_xy, end_xy, arrow_head="->", style=...)` : Straight line with arrowheads.
- `line_curved(start_xy, end_xy, bend=0.2, ...)` : Smooth bezier curves.
- `text(xy, text="...", style=...)` : Styled typography.
- `phosphor.<icon_name>(xy, width, style=...)` : Phosphor vector icons.

Chapter 8: SmartArts (Structured Infographics)

`drawlib.smartarts` automatically arranges structured visual infographics without requiring manual coordinate math for individual shapes.

8.1 Available SmartArt Components

- **`ChevronProcess`**: Sequential arrow blocks for deployment pipelines and development phases.
- **`Table`**: Structured comparison matrices with header styling and custom borders.
- **`MindMapNode`**: Radial and branching idea maps.
- **`TreeNode`**: Hierarchical organization charts and directory trees.
- **`Cycle`**: Continuous feedback loops and lifecycle workflows.
- **`Pyramid`**: Testing pyramids, security tier models, and priority hierarchies.
- **`BoxList`** / **`BulletPoints`**: Organized card stacks and bulleted summaries.
- **`GridLayout`**: Matrix-based dashboard layouts.
- **`SourceCode`**: Syntax-highlighted code blocks with line numbering.

> [!NOTE] For pointing speech bubbles and callout annotations, use `bubblespeech` from `drawlib.shapes`.

8.2 Release Pipeline and Feature Matrix Example



Category	Primary Components	Key Benefit
SmartArts	ChevronProcess, Table, MindM	No manual coordinate calculations
Diagrams	Architecture, Sequence, Flo	Standardized system architecture
Charts	Gantt, Bar, Line, Pie	Timeline and quantitative tracking

Figure 8.1: Release Pipeline and Feature Comparison Matrix

Chapter 9: Quantitative Charts

`drawlib.charts` renders engineering roadmaps, project timelines, and metric visualizations with publication quality.

9.1 Gantt Charts (`GanttChart`)

Visualize sprint schedules, milestones, and task dependencies with automated arrow routing:

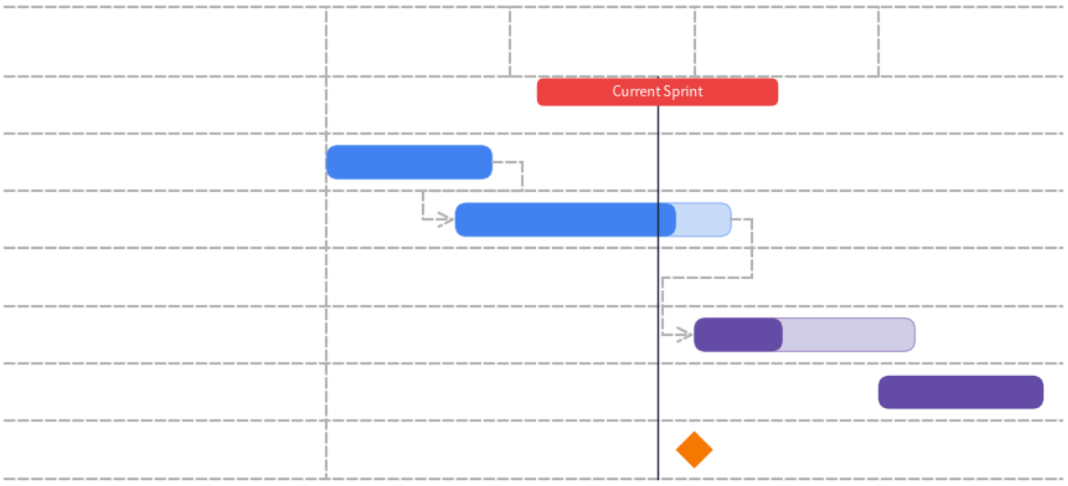


Figure 9.1: Core Platform Engineering Roadmap

9.2 Additional Chart Types

- `BarChart`: Vertical and horizontal bar plots with grouping and stacking.
- `LineChart` / `AreaChart`: Trend plots and multi-series metric tracking.
- `PieChart`: Proportional slices with smart label anti-collision.
- `RadarChart`: Multi-dimensional capability matrices.
- `ScatterChart`: Distribution plots, latency correlations, and cluster analysis.

Chapter 10: Technical Diagrams (Architecture & System Design)

`drawlib.diagrams` is the top-level module for rendering distributed architectures, microservices, and network topologies using engineering-standard notations.

10.1 Cloud Architecture Diagrams (`ArchitectureDiagram`)

Easily define boundary groups (`NodeGroup`), cloud service nodes (`Node`), and orthogonal routing connections:

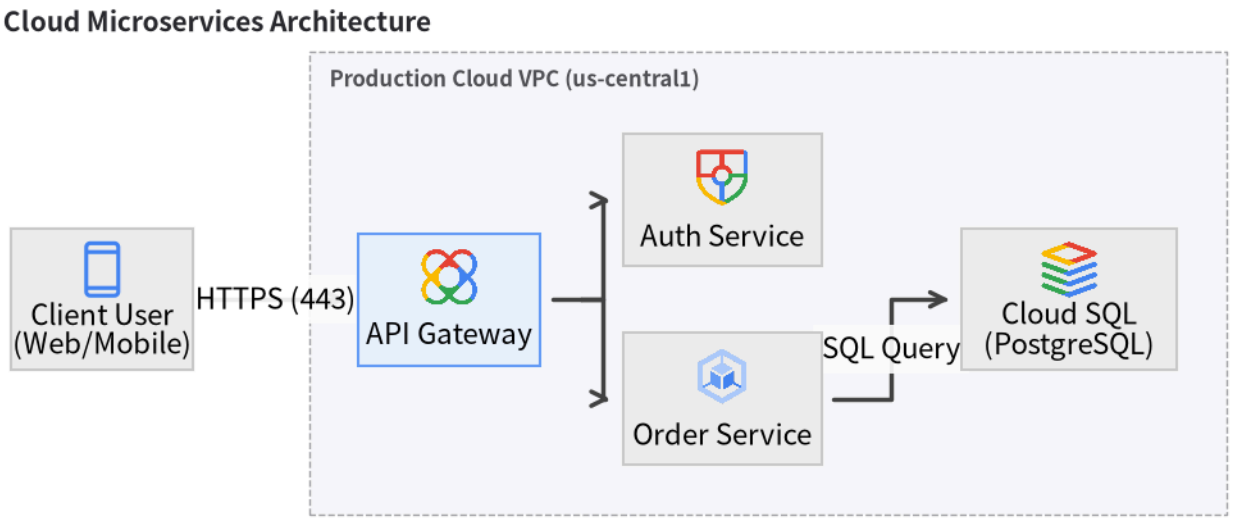


Figure 10.1: Cloud Microservices Architecture Topology

10.2 Auto-Layout Graph Engine (`drawlib.graph`)

When drafting complex architectures or data workflows from scratch, calculating pixel coordinates `(x, y)` for every node and boundary can strain cognitive working memory for both engineers and AI agents—a challenge known as the **Cold-Start Problem**.

Drawlib's `drawlib.graph` is a Pure-Python **coordinate and layout calculator (`get_layout()`)** with zero external binary dependencies (no Graphviz required). Rather than coupling layout calculation to a rigid renderer, it powers a clean **2-Step Workflow**: solve coordinates in Step 1 (`layout = g.get_layout()`), then render with `drawlib.diagrams` or `drawlib.shapes` in Step 2:

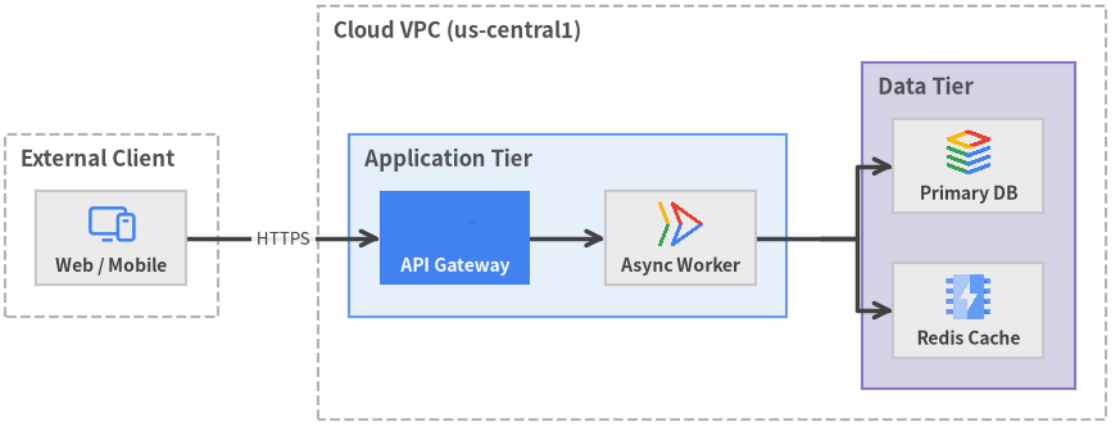


Figure 10.2: 2-Step Auto-Layout with ArchitectureGraph.get_layout() and ArchitectureDiagram

The 2-Step Workflow (`get_layout()` & `print(layout)`)

- **Step 1 — Solve & Inspect (`layout = g.get_layout()`)**: Computes node centers (`.xy` , `.local_xy`), cluster bounds (`.bottom_left` , `.local_bottom_left` , `.size`), and edge waypoints (`.points`). Call `print(layout)` to inspect the coordinate table for manual scripts, or call `layout.offset("api", dy=-4.0)` to fine-tune positions.
- **Step 2 — Render with Rich Components (`drawlib.diagrams` / `drawlib.shapes`)**: Pass the solved coordinates into `ArchitectureDiagram` , `FlowDiagram` , or `drawlib.shapes` / `drawlib.lines` for full control over icons, styles, and connectors.

5 Specialized Layout Solvers

- `ArchitectureGraph` : Multi-tier architecture, VPC/subnet boundary nesting, and 5-zone compass positioning (`left` , `right` , `top` , `bottom` , `center`).
- `LayerGraph` : Sugiyama-style hierarchical DAGs for pipelines, dataflows, and dependency graphs (`.tier()`).
- `TreeGraph` : Reingold-Tilford symmetrical tree layout for org charts, taxonomies, and ASTs (`.child()`).
- `RadialGraph` : Concentric ring layouts for hub-and-spoke and event-driven architectures (`.spoke()` , `layout.ring_radii`).
- `GridGraph` : 2D matrix grids with orthogonal channel routing for service catalogs (`.cell()` , `.cluster_row()`).

10.3 Other Domain Diagrams

- `FlowDiagram` : ISO-compliant flowcharts with decision diamonds and swimlanes.

- `SequenceDiagram`: Lifelines, synchronous/async messages, and `with d.loop()` condition frames.
- `ERDiagram`: Relational schemas with PK/FK indicators and Crow's Foot cardinalities.
- `ClassDiagram`: Object-oriented UML class diagrams with inheritance and composition.
- `StateDiagram`: Finite state machines with transition triggers and activity compartments.

Chapter 11: Style and Theme Customization

Drawlib provides complete control over document typography, color schemes, and visual styles.

11.1 Controlling Visual Themes via `styles.py`

In your project root or source directory, `styles.py` defines the active `Colors` and `Styles` namespaces:

```
from drawlib.fonts import Font
from drawlib.preset_colors import GoogleColors
from drawlib.preset_styles import GoogleStyles

# Initialize color palette and style presets
Colors = GoogleColors()
Styles = GoogleStyles().patch_font(
    regular=Font.SANSSERIF_REGULAR,
    bold=Font.SANSSERIF_BOLD,
    thin=Font.SANSSERIF_THIN,
)
```

11.2 Available Style Presets

- **`DefaultStyles`** / **`DefaultColors`**: Modern developer theme inspired by Tailwind and VitePress.
- **`GoogleStyles`** / **`GoogleColors`**: Clean editorial theme inspired by Material Design and Google Docs.
- **`MonochromeStyles`** / **`MonochromeColors`**: Grayscale theme optimized for black-and-white printing.

11.3 Deriving Custom Styles (`.patch()`)

Derive custom styles from existing presets by overriding specific attributes:

```
from drawlib.styles import Styles

# Derive a custom card style from PrimaryFlat
my_card_style = Styles.PrimaryFlat.patch(
    shape_fill_color=(235, 248, 255),
    shape_line_color=(49, 130, 206),
    shape_line_width=1.5,
)
```

11.4 Harmonizing with Document CSS (`style.css`)

When you run `drawlib init --style google`, both `styles.py` and `style.css` are configured with the Google theme. This ensures **complete visual harmony between technical text, tables, and embedded**

diagrams.

Chapter 12: Slide Deck Presentations (`slide`)

In addition to technical documents (`doc`) and multi-page documentation websites (`site`), Drawlib provides first-class support for **16:9 presentation slide decks** (`slide`).

Without relying on drag-and-drop GUI software like PowerPoint or Google Slides, you can author, version-control, and publish conference-ready slide decks directly using Markdown and declarative Python drawing code.

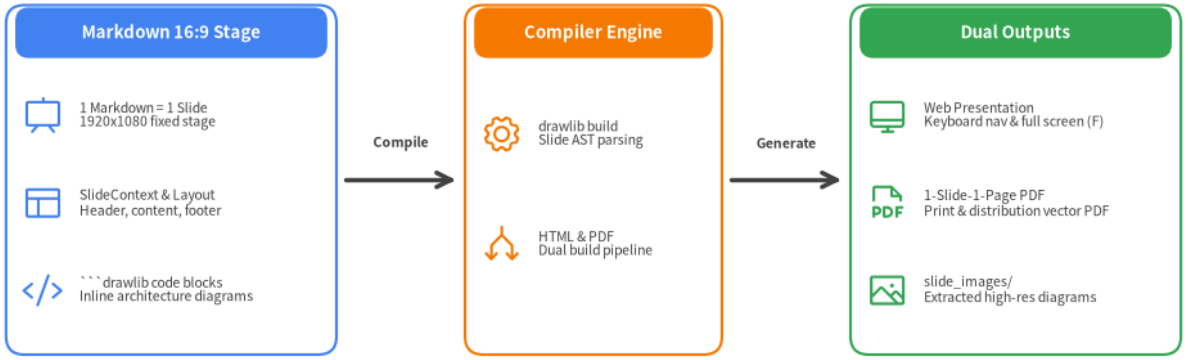


Figure 12.1: Drawlib Slide Generation Architecture and Dual Outputs

12.1 Scaffolding a Slide Project (`drawlib init slide`)

Initialize a slide presentation project with a single command:

```
# Standard slide deck in current directory (creates slide_src/)
drawlib init slide

# With custom project name, theme, and language:
drawlib init slide my_presentation -s google -l en
```

Directory Anatomy

```
my_presentation/
├── slide_src/                # [SOURCE] Slide Markdown sources and drawing code
│   ├── 01_title.md           # Title slide
│   ├── 02_agenda.md          # Agenda slide
│   ├── 03_architecture.md    # Architecture diagram slide
│   ├── styles.py             # Presentation-wide styling overrides
│   ├── utils.py              # Slide layout components and cards
│   ├── slide.js              # Keyboard navigation and presenter engine
│   ├── build.sh              # Master sequential build script
│   ├── build_html.sh         # Web presentation build script
│   ├── build_pdf.sh          # 16:9 vector PDF export script
│   └── serve.sh              # Local live-reload presentation server
├── slide_html/               # [OUTPUT] Interactive web presentation (index.html)
├── slide.pdf                 # [OUTPUT] Multi-page 16:9 vector presentation PDF
└── slide_images/             # [OUTPUT] Standalone extracted slide diagrams
```

12.2 The 16:9 Slide Authoring Model

Drawlib slide decks use an intuitive **"One Markdown File = One Slide"** convention:

- Slide sequence is determined strictly by alphabetical sorting (`01_title.md` , `02_agenda.md` , `03_overview.md` , etc.).
- Every slide is rendered onto a fixed `1920 × 1080` pixel canvas stage that automatically scales to fill any display or projector while preserving its 16:9 aspect ratio.

Slide Markdown Example

````
# Modernizing System Architecture

<div class="subtitle">Stepwise Migration to an Event-Driven Topology</div>

```drawlib width=1400px align=center file=arch_slide.png
from drawlib.canvas import setup
from drawlib.shapes import rectangle
from drawlib.lines import line
from drawlib.styles import Styles

setup(width=160, height=45)
rectangle((30, 22.5), width=35, height=18, style=Styles.Neutral, text="API Gateway")
rectangle((80, 22.5), width=35, height=18, style=Styles.PrimaryFlat, text="Event Broker",
text_style=Styles.WhiteBold)
rectangle((130, 22.5), width=35, height=18, style=Styles.SecondaryNeutral, text="Order Worker")
line((47.5, 22.5), (62.5, 22.5), arrow_head="->", style=Styles.DarkBold)
line((97.5, 22.5), (112.5, 22.5), arrow_head="->", style=Styles.DarkBold)
```
````

12.3 Dual Outputs & Presenter Experience

Compiling a `slide` project produces two presentation artifacts optimized for different contexts:

1. **Interactive Web Presentation (`slide_html/index.html`)**:
    - Designed for live presenting on projectors, external displays, or browser screen-shares.
    - **Keyboard Navigation**:
        - `Space` / `→` / `PageDown` : Advance to the next slide
        - `←` / `PageUp` : Go back to the previous slide
        - `F` : Toggle full screen
        - `O` / `Esc` : Toggle slide overview grid
2. **1-Slide-1-Page Vector PDF (`slide.pdf`)**:
    - Designed for attendee distribution, printing, or uploading to slide hosting platforms.
    - Captured via headless Chromium (Playwright), ensuring every slide is rendered with vector sharpness at true 16:9 dimensions (`@page { size: 1920px 1080px; margin: 0; }`).